ԼԱԲՈՐԱՏՈՐ ԱՇԽԱՏԱՆՔ 8

N15. Բեռնափոխադրման երթուղի

Խնդրի պահանջ

1. Կոնստրուկտոր՝ CargoTrip()՝ ընդհանուր կանոններով: Երեք պարամետրով կոնստրուկտոր՝ կոդ, ճանապարհի երկարություն, սակագին (բեռնման վճարը՝ 0), չորս պարամետրով՝ բոլոր տվյալները:
2. Ստուգումներ. երկարությունը և սակագինը դրական կամ ոչ բացասական վավեր արժեքներ են, բեռնման վճարը չի կարող բացասական լինել: Քանակները և գները պահել double տիպով:
3. Փոփոխող մեթոդ՝ ճանապարհի երկարությունը փոխելու համար:
4. Հաշվարկային մեթոդներ. հաշվել ճանապարհի արժեքը և ընդհանուր արժեքը:
5. Զանգվածի մշակում. ցուցադրել ամենաթանկ երթուղին և ընդհանուր վճարների գումարը: Որոնման ժամանակ օգտագործել համապատասխան հաշվարկային մեթոդի արդյունքը:

Աշխատանքի նպատակ

Սովորել C++ լեզվում ստեղծել դաս՝ private դաշտերով և public մեթոդներով, կիրառել կոնստրուկտորների գերբեռնում, տվյալների վավերացում, օբյեկտների զանգված և հաշվարկային մեթոդներ: Գործնականորեն հաշվել բեռնափոխադրման երթուղու արժեքը և մշակել մի քանի երթուղիների տվյալները:

Տեսական մաս

Դասը (class) միավորում է տվյալներն ու դրանց հետ աշխատող մեթոդները: private բաժինը պաշտպանում է դաշտերը անմիջական արտաքին հասանելիությունից, իսկ public բաժինը տրամադրում է օբյեկտի հետ աշխատելու միջերես:

Կոնստրուկտորը գործարկվում է օբյեկտ ստեղծելիս և սահմանում է դաշտերի սկզբնական վիճակը: Կոնստրուկտորների գերբեռնումը թույլ է տալիս նույն դասում սահմանել տարբեր պարամետրերի քանակով մի քանի կոնստրուկտոր: Յուրաքանչյուրն ընտրվում է փոխանցված արգումենտների համապատասխանությամբ:

Մեթոդը const հայտարարելը նշանակում է, որ այն չի փոխում օբյեկտի վիճակը: Տվյալների վավերացումն ապահովում է, որ հաշվարկներում չօգտագործվեն անհնար կամ բացասական արժեքներ: Երթուղու արժեքը հաշվվում է երկարությունը մեկ կմ-ի սակագնով բազմապատկելով, իսկ ընդհանուր արժեքին ավելացվում է բեռնման վճարը:

```
#include <iostream>
#include <vector>
#include <string>
#include <iomanip>
using namespace std;

class CargoTrip {
private:
    string code;
    double distanceKm;
    double ratePerKm;
    double loadingFee;

    static double validNonNegative(double value) {
        return value >= 0 ? value : 0;
    }

public:
    // Առանց պարամետրերի կոնստրուկտոր
    CargoTrip()
        : code("Unknown"), distanceKm(0),
          ratePerKm(0), loadingFee(0) {}
```

```
    // երեք պարամետրով կոնստրուկտոր
    CargoTrip(string c, double distance, double rate)
        : code(c.empty() ? "Unknown" : c),
          distanceKm(validNonNegative(distance)),
          ratePerKm(validNonNegative(rate)),
          loadingFee(0) {
        if (distance < 0 || rate < 0)
            cout << "Անվավեր արժեքը փոխարինվեց 0-ով:\\n";
    }

    // Չորս պարամետրով կոնստրուկտոր
    CargoTrip(string c, double distance,
              double rate, double fee)
        : code(c.empty() ? "Unknown" : c),
          distanceKm(validNonNegative(distance)),
          ratePerKm(validNonNegative(rate)),
          loadingFee(validNonNegative(fee)) {
        if (distance < 0 || rate < 0 || fee < 0)
            cout << "Անվավեր արժեքը փոխարինվեց 0-ով:\\n";
    }

    void setDistance(double distance) {
        if (distance >= 0) {
            distanceKm = distance;
        } else {
            cout << "երկարությունը չի կարող բացասական լինել:\\n";
        }
    }

    double routeCost() const {
        return distanceKm * ratePerKm;
    }

    double totalCost() const {
        return routeCost() + loadingFee;
    }

    void show() const {
        cout << "Կոդ՝ " << code
             << ", երկարություն՝ " << distanceKm << " կմ"
             << ", սակագին՝ " << ratePerKm
             << ", բեռնման վճար՝ " << loadingFee
             << ", ընդհանուր արժեք՝ " << totalCost()
             << '\\n';
    }

    string getCode() const { return code; }
    double getTotalCost() const { return totalCost(); }
};

int main() {
    cout << fixed << setprecision(2);

    // երեք կոնստրուկտորների օրինակներ
    CargoTrip trip1;
```

```
    CargoTrip trip2("CT-02", 80, 250);
    CargoTrip trip3("CT-03", 120, 180, 5000);

    cout << "երեք առանձին օբյեկտներ`\\n";
    trip1.show();
    trip2.show();
    trip3.show();

    // Առնվազն 5 օբյեկտից բաղկացած զանգված
    vector<CargoTrip> trips = {
        CargoTrip("CT-01", 80, 250, 3000),
        CargoTrip("CT-02", 120, 180, 5000),
        CargoTrip("CT-03", 60, 300, 2000),
        CargoTrip("CT-04", 200, 150, 4000),
        CargoTrip("CT-05", 100, 220, 2500)
    };

    // Փոփոխող մեթոդի կիրառում
    trips[0].setDistance(90);

    double totalFees = 0;
    int mostExpensive = 0;

    cout << "\\nերթուղիների ամբողջը`\\n";
    for (int i = 0; i < static_cast<int>(trips.size()); ++i) {
        trips[i].show();
        totalFees += trips[i].getTotalCost();

        if (trips[i].getTotalCost() >
            trips[mostExpensive].getTotalCost()) {
            mostExpensive = i;
        }
    }

    cout << "\\nԱմենաթանկ երթուղին` "
         << trips[mostExpensive].getCode()
         << ", արժեքը` "
         << trips[mostExpensive].getTotalCost() << '\\n';

    cout << "Բոլոր երթուղիների ընդհանուր արժեքը` "
         << totalFees << '\\n';

    return 0;
}
```